PROTOFIRE

October 2026

Smart contract audit
rain.metadata

Content

Project description **3**
Executive summary **3**
Reviews **3**
Technical analysis and findings **5**
Security findings **6**
**** Critical 6
*** High 6
** Medium 6
* Low 6
Informational 6
General Risks **7**
Approach and methodology **8**

Project description

This repository, rain.metadata, is a native Solidity library and registry system that implements the Rain Metadata V1 specification. Its primary purpose is to provide a standard, gas-efficient mechanism for associating off-chain data (such as documentation, ABIs, and source code) with on-chain artifacts.

The core component is the MetaBoard, a permissionless contract that allows any account to emit metadata events. The repository also includes LibMeta and IDescribedByMetaV1 to facilitate metadata validation, hashing, and the creation of "self-describing" contracts. This architecture enables a verifiable link between smart contracts and their metadata without incurring the high cost of on-chain storage.

Executive summary

Type	Library
Languages	Solidity
Methods	Architecture Review, Manual Review, Unit Testing, Functional Testing, Automated Review
Documentation	README.md
Repositories	https://github.com/rainlanguage/rain.metadata

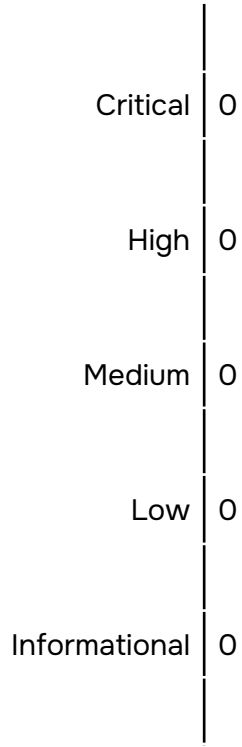
Reviews

Date	Repository	Commit
02/02/26	rain.metadata	fde2333cdf92d0879164be4eacb1eb844cea79d9
12/02/26	rain.metadata	a823381df1a118fe0e217150a416566e0c182210
29/10/26	rain.metadata	6394c9bb66583068cf1b44d1cb4ca75d0229ec81 **sol-v0.1.43 tag**

Scope

Contracts
src/lib/LibDescribedByMeta.sol
src/lib/LibIMetaBoardV1_2.sol
src/lib/LibMeta.sol
src/interface/deprecated/IMetaBoardV1.sol
src/interface/deprecated/IMetaV1.sol
src/interface/unstable/IMetaBoardV1_2.sol
src/interface/unstable/IMetaV1_2.sol
src/interface/IDescribedByMetaV1.sol

Technical analysis and findings



Security findings

**** Critical

No critical severity issue found.

*** High

No high severity issue found.

** Medium

No medium severity issue found.

* Low

No low severity issue found.

Informational

No informational severity issue found.

General Risks

- Permissionless Registry / Spam: The MetaBoard contract is fully permissionless, meaning any address can emit metadata for any subject. There is a risk of "registry poisoning" where malicious actors emit spam or misleading metadata. Off-chain indexers and tooling must implement robust filtering (e.g., verifying that the sender is authorized for the subject) to ignore untrusted data.

- Data Availability: The system enforces the integrity of the metadata (via hashing) but not its availability. Since the full metadata payload is emitted in event logs or stored off-chain, there is a risk that the underlying data becomes unavailable (e.g., if history is not preserved or off-chain storage fails), leaving only an opaque hash on-chain.

Approach and methodology

To establish a uniform evaluation, we define the following terminology in accordance with the OWASP Risk Rating
Methodology:

	**Likelihood** Indicates the probability of a specific vulnerability being discovered and exploited in real-world scenarios
	**Impact** Measures the technical loss and business repercussions resulting from a successful attack
	**Severity** Reflects the comprehensive magnitude of the risk, combining both the probability of occurrence (likelihood) and the extent of potential consequences (impact)

Likelihood and impact are divided into three levels: High H, Medium M, and Low L. The severity of a risk is a blend of these two factors, leading to its classification into one of four tiers: Critical, High, Medium, or Low.

When we identify an issue, our approach may include deploying contracts on our private testnet for validation through testing. Where necessary, we might also create a Proof of Concept PoC to demonstrate potential exploitability. In particular, we perform the audit according to the following procedure:

	**Advanced DeFi Scrutiny** We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs
	**Semantic Consistency Checks** We then manually check the logic of implemented smart contracts and compare with the description in the white paper.
	**Security Analysis** The process begins with a comprehensive examination of the system to gain a deep understanding of its internal mechanisms, identifying any irregularities and potential weak spots.